

BCSE302P — Database Systems Laboratory

DBMS Lab Project

Project Overview Document

Decentralized Surplus Food & Perishable Redistribution Platform

Challenge Track: T5 & T6 — Waste & Circular Economy / Healthcare & Well-being

Name	Ryan Fernandes
Registration No.	24BCE0565
Name	Aritra Ghosh
Registration No.	24BCE0598

Table of Contents

1. Introduction & Project Journey.....	3
2. Problem Discovery Report	3
3. Innovation Proposal	5
4. Software Requirements Specification.....	5
5. System Architecture.....	7
6. Database Design.....	7
7. Application Workflow	15
8. Testing & Validation Plan	15
9. Societal Impact & Measurable Outcomes	15
10. TRL Justification & Roadmap	16
11. Project Team	16
12. Deliverables Mapping	16

1. Introduction & Project Journey

This document presents the overview of the DBMS Lab Project undertaken for BCSE302P — Database Systems Laboratory. The chosen problem area is food wastage occurring alongside unmet nutritional need, addressed through a Decentralized Surplus Food & Perishable Redistribution Platform. The project follows the mandated journey: discover the problem, research existing practice, identify the gap, innovate a digital solution, model the database, develop a functional prototype, test and validate it, measure societal impact, and carry it through the internal review toward the October 2026 Expo/Challenge.

Project Journey followed in this document:

Discover → Research → Identify Gap → Innovate → Model Database → Develop → Test → Validate → Measure Impact → Review (10) → Expo/Challenge (Oct 2026)

2. Problem Discovery Report

2.1 Problem Statement

Large quantities of edible, still-safe surplus food are generated every day by restaurants, hostel and college mess kitchens, event caterers, wedding halls and retail/grocery stores, while a significant share of the population nearby remains food-insecure. The surplus is time-critical and perishable: it must reach a receiver within a narrow safety window, but today that connection depends on informal phone calls, personal contacts, or the surplus is simply discarded because no one nearby is discoverable in time. There is no shared, real-time, verifiable system that lets any donor publish a time-boxed surplus listing and any nearby verified receiver or volunteer discover, claim and collect it before it spoils.

2.2 Who Is Affected, and Where

Stakeholder Group	Who / Examples	Impact of the Problem
Food Donors	Restaurants, hostel/college mess kitchens, event caterers, wedding venues, bakeries, retail stores	Lose money disposing of safe surplus; face logistical/legal hesitation to donate without a trusted, timestamped process
Receivers	NGOs, shelters, old-age homes, community kitchens, verified low-income individuals/families	Face inconsistent, undiscoverable, non-real-time supply of food donations
Volunteers	Independent volunteers, student clubs, gig-delivery participants	Have no structured, geo-matched task queue for last-mile pickup and delivery
Local Administration / Municipal Bodies	Ward-level and city sanitation & welfare departments	Lack visibility into food-waste and redistribution volumes for planning and reporting

Geographic scope: the platform is designed zone-by-zone (ward / pincode-cluster level) inside a city, since surplus food redistribution is only viable within a short pickup radius and a short time window; the architecture allows the same schema to be replicated across multiple cities without a single central bottleneck, which motivates the decentralized, zone-partitioned design described in Section 5.

2.3 Supporting Evidence

Independent published data corroborates the scale and urgency of the problem in the Indian context:

- India wastes an estimated 78–80 million tonnes of food annually, the second-highest total in the world, valued at roughly ₹1.55 lakh crore, according to the UNEP Food Waste Index Report 2024.
- Per-capita household food waste in India is estimated at about 55 kg per year, most of it avoidable.
- Around 194 million people in India remain undernourished even as this volume of edible food is discarded, exposing a stark distribution gap rather than a production gap.
- Globally, food services (restaurants, caterers, institutional kitchens) contribute roughly a quarter of all food waste, a segment with far more redistribution potential than household waste because it is generated in bulk, at known times and locations.

(Sources: UNEP Food Waste Index Report 2024; India Global Hunger Index 2025 commentary. Figures are cited for problem justification; primary survey/interview evidence from local donors and NGOs is to be collected during the requirements phase as per the course guidelines.)

2.4 Existing Solutions & Gap Analysis

Existing Practice	Limitation / Gap	Proposed Innovation Response
Informal phone/WhatsApp coordination between a restaurant and a known NGO	Not discoverable beyond personal contacts; no record of quantity, timing, or outcome; breaks down when the usual contact is unavailable	Open, zone-wide discovery; anyone verified nearby can see and claim a live listing
Large centralized food-donation apps (single national NGO-run platform)	Single point of coordination and approval; onboarding is slow for small donors/local volunteers; limited real-time geo-matching; weak transparency into what happened after a claim	Any verified donor/receiver/volunteer in a zone can transact directly; matching and trust logging are automated and auditable, without waiting on a central approval desk
Municipal solid-waste / composting collection	Deals with food only after it has already become waste; does not intercept still-edible surplus	Intercepts surplus before the expiry window closes, before it becomes waste
Manual NGO food-bank ledgers (paper/Excel)	No real-time visibility, no geo-matching, error-prone, cannot enforce one-claim-per-listing safely	Database-enforced transactional claiming prevents double-allocation; live dashboards replace manual ledgers

3. Innovation Proposal

3.1 Proposed Solution Overview

The platform lets any verified Donor publish a time-boxed Food Listing (food type, quantity, preparation time, safety expiry window, geo-tagged pickup point). A Geo-Temporal Matching Engine ranks and notifies nearby verified Receivers and Volunteers based on distance, remaining time-to-expiry, and receiver capacity. The first eligible Receiver to claim a listing locks it inside a database transaction, preventing double-allocation. A Volunteer is then matched for pickup and delivery, and every state change — listed, claimed, picked up, delivered, rated — is written to a hash-chained Trust Ledger so that donors, receivers and administrators can audit exactly what happened to every unit of food, without depending on a single centrally-managed inventory.

3.2 What Makes It Different (Novelty)

Dimension	What Is New
Decentralized participation model	Any verified individual or business can act as donor, receiver, or volunteer within their zone — there is no gatekeeping onboarding queue as with a single centrally-run NGO app
Geo-temporal matching, not a static directory	Listings are ranked live by proximity and by how much of the safety window remains, so the most time-critical listings surface first instead of being buried in a generic feed
Transaction-safe claiming	A database-level transaction with row locking guarantees exactly one confirmed claim per listing, eliminating wasted trips where two receivers show up for the same food
Hash-chained trust ledger	Every listing/claim/pickup/rating event is chained with a previous-hash reference (blockchain-inspired, without the overhead of a full distributed ledger), giving tamper-evident accountability for a domain where informal trust currently has to be taken on faith
Impact analytics as a first-class output	Kilograms redistributed, meals-equivalent, and estimated CO2e avoided are computed directly from transactional data rather than self-reported after the fact

3.3 Expected Improvement

- Reduces the time between a surplus item becoming available and it being claimed, directly increasing the share of surplus that is actually redistributed before it spoils.
- Removes reliance on a single point of human coordination, so donations do not fail simply because ‘the usual contact’ is unreachable.
- Gives administrators and researchers a verifiable, query-able dataset of redistribution activity for planning, reporting and eventually policy use.
- Creates measurable accountability (ratings + trust ledger) that can increase donor willingness to participate, which is currently a documented barrier to food donation.

4. Software Requirements Specification

4.1 Stakeholders / System Actors

- Donor — restaurant, mess, caterer, retail store, or individual with surplus food to give away
- Receiver — NGO, shelter, community kitchen, or verified individual who claims listings
- Volunteer — performs pickup and delivery between donor and receiver
- Zone Coordinator / Admin — verifies users, monitors zone activity, resolves disputes

4.2 Functional Requirements

1. Users can register and be verified into a Role (Donor / Receiver / Volunteer / Admin) and a Zone.
2. Donors can create, update and cancel a Food Listing with quantity, category, preparation time and an expiry window.
3. The system automatically computes and displays remaining time-to-expiry and flags listings approaching expiry.
4. Receivers can view a ranked, zone-filtered feed of live listings and claim one; the claim must be transaction-safe so a listing cannot be claimed twice.
5. Volunteers can accept a pickup task tied to a confirmed claim and update pickup/delivery status and timestamps.
6. Both parties can rate each other after a completed transaction; ratings feed into a user trust score.
7. Every state-changing action is appended to an auditable Trust Ledger with a hash reference to the previous entry for that user.
8. The system sends notifications (push/SMS) to matched receivers and volunteers when a relevant listing appears or a status changes.
9. Admins/Zone Coordinators can view dashboards of listings, claims, pickups, and aggregate impact metrics per zone and city.
10. The system supports reporting: total kilograms redistributed, estimated meals served, and estimated CO2e avoided, filterable by date range and zone.

4.3 Non-Functional Requirements

Requirement	Description
Data Integrity	Claim confirmation must be ACID-compliant so a listing is never allocated to more than one receiver
Performance	Nearby-listing queries should return results within a couple of seconds even as listing volume grows, aided by spatial and expiry-window indexing
Availability	Zone-partitioned data model should allow one zone's load or outage to not block another zone's transactions
Security & Privacy	Role-based access control; location data limited to what is needed for matching; passwords hashed, never stored in plain text
Auditability	Every critical action must be traceable through the Trust Ledger for dispute resolution
Scalability	Schema and matching logic should scale from a single-city pilot to a multi-city deployment without redesign

5. System Architecture

The system follows a layered architecture: role-specific client interfaces talk to a REST API gateway that enforces role-based access control; the application layer hosts the Listing, Matching, Trust & Verification, Notification and Analytics services; all services read and write through a single PostgreSQL database extended with PostGIS for spatial queries. The design is decentralized at the participation level (any verified party can transact directly within a zone without central approval) and zone-partitioned at the data level, so the same schema can be replicated per city.

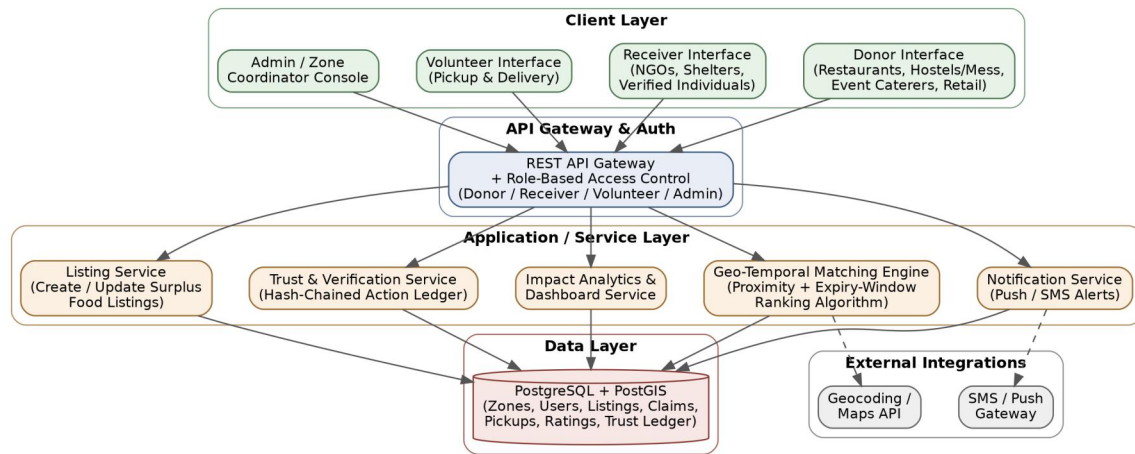


Figure 1: System Architecture — Client, Gateway, Service and Data layers

6. Database Design

The database is the central technical component of this project, as required by the course. This section presents the ER/EER model, the derived relational schema with keys and constraints, the normalization justification, and representative DDL/DML and programmability features (triggers, procedures, views, transactions, indexing).

6.1 ER / EER Model

Eight entities are modelled: ZONE, USERS (a single actor table distinguished by a Role attribute, used because Donor/Receiver/Volunteer/Admin share the same core attributes and a person may hold more than one role over time), FOOD_LISTING, CLAIM, PICKUP (a weak entity identified together with its owning CLAIM), RATING, NOTIFICATION and TRUST_LEDGER. RATING models a many-to-many self-referencing relationship on USERS through two foreign keys, RaterID and TargetUserID, both scoped to a specific CLAIM.

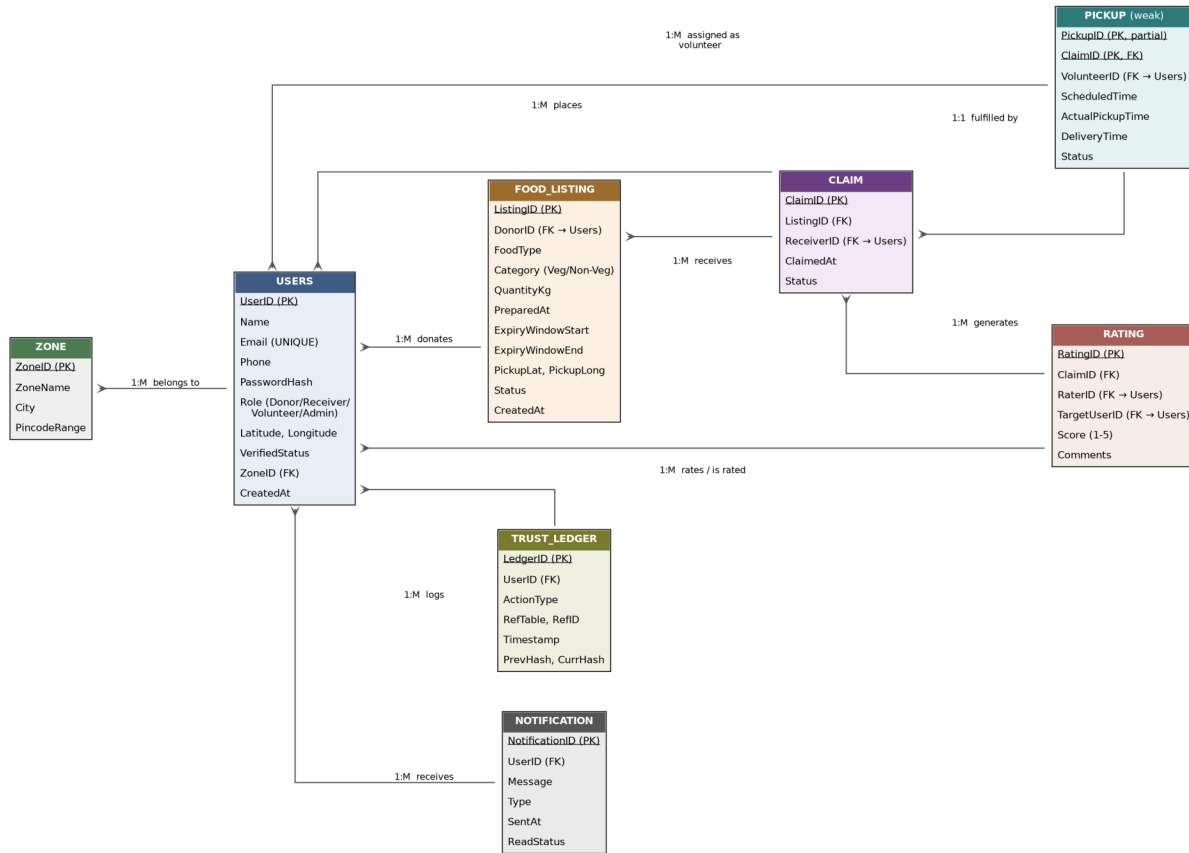


Figure 2: ER / Relational Schema Diagram with primary keys, foreign keys and cardinalities

Entity Notes

- **USERS — ZONE** is a 1:M owner relationship (a zone has many users); Role is an enumerated attribute rather than a subtype table, chosen after weighing normalization purity against the practical need for a person to hold multiple roles without duplicate rows.
- **FOOD_LISTING** — 1:M from **USERS** (as Donor); ExpiryWindowStart/End are the attributes the Matching Engine and expiry triggers operate on.
- **CLAIM** — 1:M from **FOOD_LISTING** and 1:M from **USERS** (as Receiver); a UNIQUE constraint together with an application-level transaction ensures at most one CONFIRMED claim per listing.
- **PICKUP** — a weak entity; it cannot exist without a CLAIM and is identified by (ClaimID, PickupID).
- **RATING** and **TRUST_LEDGER** — both reference **USERS** and **CLAIM** to keep every accountability record traceable to a specific transaction.

6.2 Relational Schema

The ER model above is mapped to the following relations (primary keys underlined in the diagram; PK/FK/constraints listed per table below):

ZONE

Attribute	Type	Constraint
ZoneID	INT	PRIMARY KEY
ZoneName	VARCHAR(60)	NOT NULL

Attribute	Type	Constraint
City	VARCHAR(60)	NOT NULL
PincodeRange	VARCHAR(20)	NOT NULL

USERS

Attribute	Type	Constraint
UserID	INT	PRIMARY KEY
Name	VARCHAR(100)	NOT NULL
Email	VARCHAR(100)	UNIQUE, NOT NULL
Phone	VARCHAR(15)	UNIQUE, NOT NULL
PasswordHash	VARCHAR(256)	NOT NULL
Role	ENUM(Donor,Receiver,Volunteer,Admin)	NOT NULL
Latitude	DECIMAL(9,6)	NOT NULL
Longitude	DECIMAL(9,6)	NOT NULL
VerifiedStatus	BOOLEAN	DEFAULT FALSE
ZoneID	INT	FOREIGN KEY → ZONE(ZoneID)
CreatedAt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

FOOD_LISTING

Attribute	Type	Constraint
ListingID	INT	PRIMARY KEY
DonorID	INT	FOREIGN KEY → USERS(UserID)
FoodType	VARCHAR(80)	NOT NULL
Category	ENUM(Veg,NonVeg)	NOT NULL
QuantityKg	DECIMAL(6,2)	CHECK (QuantityKg > 0)
PreparedAt	TIMESTAMP	NOT NULL
ExpiryWindowStart	TIMESTAMP	NOT NULL
ExpiryWindowEnd	TIMESTAMP	CHECK (ExpiryWindowEnd > ExpiryWindowStart)
PickupLat / PickupLong	DECIMAL(9,6)	NOT NULL
Status	ENUM(Available,Claimed,PickedUp,Expired,Cancelled)	DEFAULT 'Available'
CreatedAt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

CLAIM

Attribute	Type	Constraint
ClaimID	INT	PRIMARY KEY

Attribute	Type	Constraint
ListingID	INT	FOREIGN KEY → FOOD_LISTING(ListingID)
ReceiverID	INT	FOREIGN KEY → USERS(UserID)
ClaimedAt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
Status	ENUM(Pending,Confirmed,Cancelled)	DEFAULT 'Pending'

PICKUP (weak entity)

Attribute	Type	Constraint
PickupID	INT	PARTIAL KEY
ClaimID	INT	PRIMARY KEY (composite), FOREIGN KEY → CLAIM(ClaimID)
VolunteerID	INT	FOREIGN KEY → USERS(UserID)
ScheduledTime	TIMESTAMP	NOT NULL
ActualPickupTime	TIMESTAMP	NULLABLE
DeliveryTime	TIMESTAMP	NULLABLE
Status	ENUM(Scheduled,PickedUp,Delivered,Missed)	DEFAULT 'Scheduled'

RATING

Attribute	Type	Constraint
RatingID	INT	PRIMARY KEY
ClaimID	INT	FOREIGN KEY → CLAIM(ClaimID)
RaterID	INT	FOREIGN KEY → USERS(UserID)
TargetUserID	INT	FOREIGN KEY → USERS(UserID)
Score	SMALLINT	CHECK (Score BETWEEN 1 AND 5)
Comments	VARCHAR(300)	NULLABLE

TRUST_LEDGER

Attribute	Type	Constraint
LedgerID	INT	PRIMARY KEY
UserID	INT	FOREIGN KEY → USERS(UserID)
ActionType	VARCHAR(40)	NOT NULL
RefTable / RefID	VARCHAR(40) / INT	NOT NULL
Timestamp	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
PrevHash / CurrHash	CHAR(64) / CHAR(64)	NOT NULL

NOTIFICATION

Attribute	Type	Constraint
NotificationID	INT	PRIMARY KEY
UserID	INT	FOREIGN KEY → USERS(UserID)
Message	VARCHAR(200)	NOT NULL
Type	VARCHAR(30)	NOT NULL
SentAt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
ReadStatus	BOOLEAN	DEFAULT FALSE

6.3 Normalization Justification

The schema is designed directly in Third Normal Form (3NF); the reasoning below documents how partial and transitive dependencies were avoided rather than treating normalization as an afterthought.

- 1NF: every attribute is atomic — for example PickupLat/PickupLong are stored as two separate decimal columns rather than one combined 'location' string, and a listing's category is a single enumerated value rather than a repeating group.
- 2NF: all non-key attributes depend on the whole primary key. This is most visible in PICKUP, a weak entity whose key is the composite (ClaimID, PickupID) — ScheduledTime and Status depend on the pickup event as a whole, not on ClaimID alone, which is why Pickup was not folded directly into Claim.
- 3NF: no non-key attribute depends on another non-key attribute. Donor contact details are not repeated inside FOOD_LISTING (they are looked up via DonorID → USERS); Zone name/city are not repeated inside USERS (looked up via ZoneID → ZONE). This removes update anomalies — correcting a donor's phone number or a zone's city updates one row instead of every listing or user row that mentions it.
- Design trade-off considered: USERS could have been split into DonorProfile / ReceiverProfile / VolunteerProfile subtype tables (a stricter 3NF/BCNF decomposition for role-specific attributes such as an NGO's registration number). This was deferred to a documented extension because the core transactional flow (list → claim → pickup → rate) does not require role-specific attributes yet; it is flagged in the roadmap in Section 10.

6.4 Sample DDL

```
CREATE TABLE Zone (
  ZoneID      SERIAL PRIMARY KEY,
  ZoneName    VARCHAR(60) NOT NULL,
  City        VARCHAR(60) NOT NULL,
  PincodeRange VARCHAR(20) NOT NULL
);

CREATE TABLE Users (
  UserID      SERIAL PRIMARY KEY,
  Name        VARCHAR(100) NOT NULL,
  Email       VARCHAR(100) UNIQUE NOT NULL,
  Phone       VARCHAR(15)  UNIQUE NOT NULL,
  PasswordHash VARCHAR(256) NOT NULL,
  Role        VARCHAR(12) CHECK (Role IN ('Donor', 'Receiver', 'Volunteer', 'Admin')),
  Latitude    DECIMAL(9,6) NOT NULL,
  Longitude   DECIMAL(9,6) NOT NULL,
  VerifiedStatus BOOLEAN DEFAULT FALSE,
  ZoneID      INT REFERENCES Zone (ZoneID),
  CreatedAt   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```

CREATE TABLE Food_Listing (
    ListingID          SERIAL PRIMARY KEY,
    DonorID            INT REFERENCES Users(UserID),
    FoodType           VARCHAR(80) NOT NULL,
    Category           VARCHAR(10) CHECK (Category IN ('Veg','NonVeg')),
    QuantityKg         DECIMAL(6,2) CHECK (QuantityKg > 0),
    PreparedAt         TIMESTAMP NOT NULL,
    ExpiryWindowStart  TIMESTAMP NOT NULL,
    ExpiryWindowEnd    TIMESTAMP NOT NULL CHECK (ExpiryWindowEnd > ExpiryWindowStart),
    PickupLat          DECIMAL(9,6) NOT NULL,
    PickupLong         DECIMAL(9,6) NOT NULL,
    Status             VARCHAR(12) DEFAULT 'Available',
    CreatedAt          TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Claim (
    ClaimID            SERIAL PRIMARY KEY,
    ListingID          INT REFERENCES Food_Listing(ListingID),
    ReceiverID         INT REFERENCES Users(UserID),
    ClaimedAt          TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    Status             VARCHAR(12) DEFAULT 'Pending'
);

CREATE TABLE Pickup (
    PickupID           SERIAL,
    ClaimID            INT REFERENCES Claim(ClaimID),
    VolunteerID        INT REFERENCES Users(UserID),
    ScheduledTime       TIMESTAMP NOT NULL,
    ActualPickupTime    TIMESTAMP,
    DeliveryTime        TIMESTAMP,
    Status             VARCHAR(12) DEFAULT 'Scheduled',
    PRIMARY KEY (ClaimID, PickupID)
);

```

6.5 Database Programmability — Queries, Transactions, Triggers, Views, Indexing

Transaction-safe claiming (concurrency control)

Confirming a claim must be atomic so two receivers can never both succeed against the same listing. This is enforced with an explicit transaction and row-level locking:

```

BEGIN;
SELECT Status FROM Food_Listing WHERE ListingID = :id FOR UPDATE;
-- application checks Status = 'Available'
UPDATE Food_Listing SET Status = 'Claimed' WHERE ListingID = :id;
INSERT INTO Claim (ListingID, ReceiverID, Status) VALUES (:id, :receiverId, 'Confirmed');
COMMIT;

```

Trigger: auto-expire stale listings

```

CREATE OR REPLACE FUNCTION expire_stale_listing() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.ExpiryWindowEnd < CURRENT_TIMESTAMP AND NEW.Status = 'Available' THEN
        NEW.Status := 'Expired';
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_expire_listing
BEFORE UPDATE ON Food_Listing
FOR EACH ROW EXECUTE FUNCTION expire_stale_listing();

```

Nested / aggregate query — nearest live listings ranked by urgency

```

SELECT l.ListingID, l.FoodType, l.QuantityKg,
       EXTRACT(EPOCH FROM (l.ExpiryWindowEnd - now()))/60 AS minutes_left,
       ROUND((6371 * acos(cos(radians(:lat)) * cos(radians(l.PickupLat))
       * cos(radians(l.PickupLong) - radians(:lng))
       + sin(radians(:lat)) * sin(radians(l.PickupLat))))::numeric, 2) AS distance_km
FROM Food_Listing l
WHERE l.Status = 'Available'
      AND l.ListingID NOT IN (SELECT ListingID FROM Claim WHERE Status = 'Confirmed')
ORDER BY minutes_left ASC, distance_km ASC
LIMIT 20;

```

View — zone-level impact dashboard

```

CREATE VIEW Zone_Impact_Summary AS
SELECT z.ZoneName,
       COUNT(DISTINCT fl.ListingID) AS total_listings,
       COUNT(DISTINCT c.ClaimID) FILTER (WHERE c.Status='Confirmed') AS total_claims,
       SUM(fl.QuantityKg) FILTER (WHERE fl.Status='PickedUp') AS kg_redistributed,
       ROUND(SUM(fl.QuantityKg) FILTER (WHERE fl.Status='PickedUp') / 0.4, 0) AS
meals_equivalent
FROM Zone z
JOIN Users u ON u.ZoneID = z.ZoneID
JOIN Food_Listing fl ON fl.DonorID = u.UserID
LEFT JOIN Claim c ON c.ListingID = fl.ListingID
GROUP BY z.ZoneName;

```

Indexing and access control notes

- A composite index on Food_Listing(Status, ExpiryWindowEnd) and a spatial (PostGIS GiST) index on the pickup coordinates keep the nearest-and-most-urgent query fast as listing volume grows.
- A unique partial index on Claim(ListingID) WHERE Status='Confirmed' gives the database itself a second guarantee, beyond the transaction, that a listing can never carry two confirmed claims.
- Role-based access is enforced at the application layer and mirrored with PostgreSQL row-level security policies so a Receiver's session can never read another user's PasswordHash or another donor's unlisted drafts.

7. Application Workflow

The end-to-end workflow ties the database transactions above into a single user-facing journey, from a donor listing surplus food through to impact measurement:

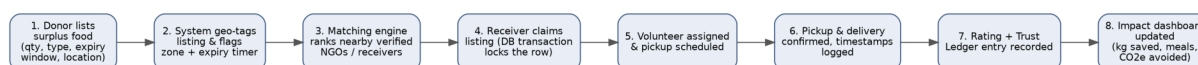


Figure 3: End-to-end redistribution workflow

8. Testing & Validation Plan

Testing is planned at three levels, aligned with the mandatory deliverables:

8.1 Database-level testing

- Constraint tests: attempt inserts that violate CHECK/UNIQUE/FK constraints (e.g. negative QuantityKg, ExpiryWindowEnd before Start) and confirm rejection.
- Concurrency test: fire two simultaneous claim transactions against the same listing and confirm exactly one commits.
- Trigger test: advance a listing past ExpiryWindowEnd and confirm the trigger flips Status to 'Expired'.

8.2 Application-level testing

- Functional test cases for each requirement in Section 4.2 (create listing, claim, assign volunteer, rate, notify).
- Realistic scenario simulation: seed the database with a week of representative listings across 3–4 zones (varying food types, quantities and expiry windows) drawn from patterns reported by local restaurants/mess kitchens and NGOs, and measure claim latency and match quality.

8.3 Validation against real stakeholders

- Short structured interviews / a small survey with at least one campus mess or local eatery (potential donor) and one local NGO or shelter (potential receiver) to validate the workflow and the required fields before the internal review.
- Compare pre- and post-pilot informal donation rates, if a short pilot window is feasible, as evidence for the Impact & TRL Report.

9. Societal Impact & Measurable Outcomes

Impact is computed directly from transactional data via the Zone_Impact_Summary view (Section 6.5) rather than self-reported afterwards, so it is auditable:

Metric	How It Is Computed
Kilograms of food redistributed	SUM(QuantityKg) over listings with Status = 'PickedUp', per zone and time period
Meals-equivalent served	Kilograms redistributed ÷ standard meal weight (e.g. 0.4 kg/meal), a widely used FAO-style conversion
Estimated CO2e avoided	Kilograms redistributed × an emission-factor constant for landfill food waste, reported as an estimate

Metric	How It Is Computed
Donor and receiver participation growth	COUNT(DISTINCT UserID) by role and zone over time
Average time-to-claim	AVG(ClaimedAt – CreatedAt) per listing, an indicator of matching efficiency

At scale, this dataset can also support the wider policy goal identified in national commentary on food waste — that a consolidated, database-backed record of institutional food waste and redistribution is currently missing and has been explicitly called for.

10. TRL Justification & Roadmap

The project targets TRL 4 by the internal review — a working prototype with the full database schema, transaction-safe claiming, and core workflows validated in a controlled environment using seeded realistic data — advancing toward TRL 5 by the Expo, where the prototype is validated with real donor/receiver participants in at least one campus or neighbourhood zone.

Phase	Target TRL	Evidence / Milestone
Now → Internal Review	TRL 3–4	Schema implemented and populated; core transactions, trigger and matching query working against realistic seeded data
Internal Review → Expo (Oct 2026)	TRL 4–5	Pilot with at least one real donor and one real receiver/NGO in a single zone; live claim-to-delivery cycle demonstrated
Post-Expo roadmap	TRL 5+	Multi-zone rollout, DonorProfile/ReceiverProfile subtype tables, PostGIS-backed multi-city partitioning, SMS gateway integration for donors without smartphones

11. Project Team

Team size is capped at 3 students per the course guidelines; the team currently has 2 members, with every member assigned an identifiable contribution across problem analysis, database design, development, testing and presentation.

Name	Registration No.	Primary Contribution Area
Ryan Fernandes	24BCE0565	Database design, backend/matching logic, documentation
Aritra Ghosh	24BCE0598	Frontend / application prototype, testing & validation

12. Deliverables Mapping

Mapping this document and planned artifacts against the eight mandatory deliverables:

Deliverable	Status / Location
1. Problem Discovery Report	Section 2 of this document
2. Innovation Proposal	Section 3 of this document
3. Software Requirements	Section 4 of this document
4. Database Design	Section 6 of this document (ER/EER, schema, normalization, DDL)
5. Application Prototype	In development — Listing, Claim, Pickup and Dashboard modules per Section 5 architecture
6. Testing & Validation	Section 8 plan; results to be appended after execution
7. Impact & TRL Report	Sections 9–10 of this document; to be updated with pilot data
8. Expo Presentation	To be prepared for October 2026, built from this overview document